Nonlinear Dynamics in Computer Graphics: Reaction–Diffusion Systems in Procedural Generation

Ragav Srinivasan

November 9, 2026

Abstract

We start by ti

1 Introduction

1.1 Structure of this report

Section 2 gives the background on reaction–diffusion systems and the phenomena they model. We then discuss a famous result of Alan Turing on reaction–diffusion systems, and finally focus on a simple reaction–diffusion system called the Gray–Scott model. Section 3 is my original contribution: a stability analysis of the Gray–Scott model and a numerical simulation of it. Section 4 gives the conclusion and possible extensions of this project.

2 Background

2.1 Procedural Generation

- Procedural Generation is a technique in Computer Graphics that generates smooth textures and landscapes in real time. Artists and Developers can create *lifelike* 3D terrains with mountains, rivers and vegetation without the need to design or program them manually. To the player, these terrains will generate on the fly no pre-processing required.
- The most famous use case of Procedural Generation can be found in Minecraft which makes heavy use of procedural techniques to generate worlds and cave systems.
- Procedural generation is a broad technique that encompasses many algorithms. However, we will particularly looking at one specific algorithm: *Perlin Noise*.

Perlin Noise was devised by Ken Perlin at The Mathematical Animation Group while working for the 1982 Disney Sci-fi film *Tron*. Perlin's noise function still sees widespread use even in modern day games.

[TODO: Maths behind perlin noise]

$$noise(x) = x$$

- Perlin noise is a continuous function that uses

Textures created by Perlin Noise and other noise functions are random and lack structure Lagae et al. 2010. This is a useful feature for landscapes but if we wanted a zebra whose stripes have self organized patterns, then a different approach is in order.

2.2 Reaction Diffusion Systems

This brings us to the heart of this report, *Reaction Diffusion Systems*. These are Nonlinear dynamical systems originating from the world of Chemistry that models the intereactions between multiple chemical susbtances and their diffusivity with resepct to space and time.
Diffusion refers to the spread of chemical substances between low and high concentration. It turns out adding an element of diffusion leads to some very interesting pattern formation.
The General Form of Reaction Diffusion Systems is given by:

$$\frac{\partial \mathbf{q}}{\partial t} = D\nabla^2 \mathbf{q} + R(\mathbf{q}), \tag{2.1}$$

- The equation is *dimensionless*
- $\mathbf{q}$ is a vector of unknown vector functions representing the concentrations of an array of chemical substances.
- D gives the diffusivity for each chemical
- $R(\mathbf{q})$ is another vector functions that describes all the chemical reactions.

There are several systems that fit the criteria of Reaction Diffusion Systems. Notably, the activator eqution, some russian equation, etc.

2.3 Turing patterns

[TODO: Define Turing patterns]
Turing patterns are self organized patterns that resembles structures found in nature.
[TODO: Cite turing's 1952 discovery on chemical morphogenesis]

2.4 The Gray-Scott model

The Gray Scott model is a simple reaction diffusion system that models the interaction between chemicals U and V based on the following rule:

$$\begin{aligned} U + 2V &\to 3V \\ V &\to P \end{aligned}$$

$$\begin{aligned} \frac{\partial U}{\partial t} &= -UV^2 + D_u\nabla^2 + f(1-U) \\ \frac{\partial V}{\partial t} &= UV^2 + D_v\nabla^2 + (k+f)V \end{aligned}$$

- D_u and D_v is the diffusivity of chemicals U and V.
- U and V are the unknown vector functions representing the concentrations of the respective chemicals in space and time.
- f is the feed rate of chemical U for V
- k is the decay rate for chemical V.

The broader goal of this project is to explain pattern formation rising from Reaction Diffusion Systems but the narrower goal is to find which parameter values of F and k. Therefore, In the end we will build a parameter map of the Gray Scott Model that can identify which patterns we get.

3 Original Contribution

3.1 Stability analysis

A stability analysis gives us a guess on the behaviour of the system. For instance, a system is said to be stable if solutions move away from the equillibrium point, otherwise an unstable system is when solutions move away from the equillibrium point.

The point of this stability analysis is to show how adding a diffusion effect can destabilize a system. This is also called as Turing instability.

In this case, we will be doing a stability analysis twice. The first part will prove the Gray Scott Model is stable when we drop diffusion. In the second part we will show that the Gray Scott Model's turing unstable after adding the diffusion constant.

The steps of the stability analysis are:

- Drop the diffusion terms and study the spatially uniform states of the system.
- Solve the system for its equilibrium points.
- Classify each equilibrium point by substituting it into the Jacobian: is it stable or unstable?
- Use bifurcation analysis to find where, in the (F, k) plane, the equilibria appear or change stability.
- Add the diffusion terms back and ask whether a stable uniform state becomes unstable. If it does, find which parameters F and k result in Turing patterns.

  The jacobian is the matrix of partial derivatives.

$$J = \begin{pmatrix} U_{xx}, U_{xy} \\ U_{xy}, U_{yy} \end{pmatrix}$$

3.2 Numerical simulation of the Gray-Scott model

This subsection documents my numerical simulation of the Gray–Scott model. The simulation uses a finite difference approximation based on a 5-point stencil to approximate the Laplacian at each cell of a 20×20 grid. The code for the simulation is in the appendix. [TODO: Method details, figures and discussion.]

4 Conclusion

[TODO: Summarise what the stability analysis predicts, how well the simulation matches it, and what this means for someone choosing F and k to generate textures. Possible extensions: varying the diffusion ratio D_u/D_v, comparing the predicted and simulated spot spacing, and running the simulation on a 3D surface as in Turk (1991).]

References

Lagae, Ares et al. (2010). "A Survey of Procedural Noise Functions". In: *Computer Graphics Forum* 29.8, pp. 2579–2600. DOI: 10.1111/j.1467-8659.2010.01827.x.

Turk, Greg (1991). "Generating Textures on Arbitrary Surfaces Using Reaction-Diffusion". In: *Computer Graphics (Proceedings of SIGGRAPH '91)* 25.4, pp. 289–298. DOI: 10.1145/127719.122749.

Procedural Generation is a broad area in Computer Graphics that involves the generation of 3D terrains Lagae et al. 2010. This lets artists and game developers can create realistic 3D terrains

with mountains, rivers and vegetation without the need to design or program them manually. To the player, these terrains will generate on the fly with no pre-processing required. The most famous use case of procedural generation can be found in the video game Minecraft which uses it heavily.

Procedural Generation is a broad area in Computer Graphics that involves the generation of 2D and 3D textures in real time. Artists and developers can create rali

5 Appendix